

Agent Harness 详解：

以 OpenCode 与 Claude Code · Codex · DeepSeek Harness 为例

Context · Memory · Agent Loop · Multi-Agent · Tool · 自进化

「Prompts are suggestions. Harness code is law.」

—— Mitchell Hashimoto · HashiCorp 联合创始人，Vagrant / Terraform 作者

基础设施软件领域的代表工程师，近年持续写作 AI 编程 Agent 与 Harness 主题

演进背景：从 Prompt 到 Context，再到 Harness

Prompt Engineering

2022 — 2024

如何让模型理解意图

关注：单次输出质量

优化单次 LLM 调用

Zero-shot / Few-shot / CoT

硬边界：只优化"单轮"

Context Engineering

2025

如何给出正确的知识边界

关注：给模型看什么信息

管理整个上下文信息空间

策展 · 最优 · 演化

目标：最小 Token × 高信噪比

Harness Engineering

2026 —

如何让 Agent 可靠、不失控

关注：长周期任务的可靠性

围绕大模型的确定性基础设施

四层架构 · 约束边界

把错误转为可修复的系统漏洞

三者的关系与 Harness 的定位

叠加关系：Harness $\supseteq$ Context $\supseteq$ Prompt——每层解决上一层遗留的问题，不是替代

定义：围绕大模型的确定性基础设施，让非确定性模型在生产中相对确定、可控

业界数据：同模型仅换 Harness, Terminal Bench 2.0 通过率 52.8% $\rightarrow$ 66.5% · 纵深：Loop / Graph Eng.

两条核心公式

生产级 Agent = 模型潜能 - 模型熵增 + Harness 约束

最优上下文 = 最小 Token × 最高信噪比

为什么模型「不确定」

本质是 next-token 概率预测，靠采样生成
同一输入多次调用，结果可能不同
对输入措辞敏感，可能产生幻觉
采样参数：temperature / top-p / 随机种子
→ 即「熵增」：行为可能漂移

为什么 Harness「确定」

由确定性代码构成：状态机、规则引擎
同样输入 → 同样输出，可重复执行
可测试、可回滚、可观测
权限、重试、超时均为硬约束
→ 即「熵减」：用确定性框住不确定性

来源 最优上下文公式 —— Anthropic 《Effective Context Engineering for AI Agents》· 2025.09

熵增 / 熵减框架 —— 业界 Harness Engineering 工程实践总结，我们内部优化的分析框架

模型负责「生成」，Harness 负责「约束」——每一 token 都要值得它的位置。

CHAPTER 01

第一章 · 通用 Harness 架构

四层架构

六项目通用抽象：Claude Code / OpenCode / Codex / Hermes / DSH / Pi

六项目横向对比

一张表看懂共性与差异 · 差异决定选型

阅读主线

先通用机制 → 六项目对照 → 逐章展开工程要点

通用 Harness 四层架构：六项目共同抽象

L1 推理与编排层 —— 决定怎么循环

重点：QueryEngine · queryLoop · TAOR · multi-Agent

QueryEngine 会话层：submitMessage 接收消息 → 构建 SystemPrompt → 处理用户输入 → recordTranscript 持久化

queryLoop 执行层：LLM 流式调用 → 状态机 → 工具执行 | multi-Agent：Subagents / Swarm / Coordinator

L2 上下文与记忆层 —— 决定看什么信息

重点：Context 上下文管理 · Memory 系统（四分类 · AutoDream · CLAUDE.md / MEMORY.md）

Context：静态段 + 动态段缓存 · 四阶段流水线压缩

Memory：CLAUDE.md 记忆文件 · MEMORY.md 索引 · AutoDream 闲置时后台总结

L3 工具与安全执行层 —— 决定能做什么

重点：Tool 工具系统 43+ · command 命令系统 100+ · 权限引擎 · 系统级沙箱

Tool：BashTool · File Read/Write · Agent Tool · MCP · Skill · Grep | command：/help /init /review /plan /diff /cost

安全控制：权限引擎 · 系统级沙箱 · Bash AST 分析 · 自动意图分类器

L4 支撑与基础层 —— 决定跑得稳吗

重点：多端交互入口 · AgentSDK · Session · Provider · 可观测性

交互层：CLI · TUI · Desktop · Web · IDE 插件 · AgentSDK（异步生成器事件流 · 同进程异步调用）

Session 会话 / Provider 模型接入 / Config 配置 / 可观测性

Claude Code 实际架构与四层抽象——对应 · 六项目共性骨架

六项目横向对比：共性与差异

同一模型放进不同 Harness，能力表现不同 · 差异主要在记忆、权限、多 Agent 与自进化的取舍

维度	Claude Code	OpenCode	DSH	Codex	Pi	Hermes
语言	TS/Bun	TS/Bun	TS/Bun	Rust	TS	Python
架构风格	模块化服务	事件驱动	插件微内核	Thread/Turn	极简循环	自进化闭环
工具数	43+	16-18	40+	视配置	4	40+
压缩触发	窗口-13K	count≥usable	双触发	90% 双 scope	-16384	×50%
记忆	memdir+索引	无长期记忆	无长期记忆	SQLite+git	无	MEMORY+FTS5
权限	6级+AST+分类器	三层 Ruleset	守卫+ToolGuard	Approval+Sandbox	无	写审批三态
自进化	autoDream	无	无(可组合)	两阶段管线	无	curator+RL
多 Agent	Task+SendMessage	Task 子代理	6 provider	Thread fork	无	子Agent禁递归

六项目一句话介绍见讲稿 · 差异集中在记忆 / 多 Agent / 自进化三行

共性：四层架构 · 六步循环 · 压缩机制 · fail-closed 权限——差异在后文逐章展开

CHAPTER 02

第二章 · Context System

三大约束

Lost in the Middle · Context Rot · $O(n^2)$

压缩机制

触发 → 阈值 → 重组 全流程

瘦身实践

每轮固定上下文典型可减半（机制见后）

为什么信息越多，Agent 反而越差

三条经过实验验证的约束，构成 Context Engineering 的理论基础

① Lost in the Middle

位置衰减 · 效果问题：漏答

开头 75% · 结尾 75%

中间 35% ↓

机制：关键信息居中被忽视
(Stanford 2023 · U 型曲线)

对策：关键信息放首尾
长文档引用锚定位

影响：漏答 · 中段信息盲区

② Context Rot

上下文腐化 · 效果问题：答得差

1k → 4k → 16k → 64k

质量 ↓↓

机制：无关内容稀释有效信号
(Chroma 2025 · 与长度相关)

对策：压缩 · 折叠
只留相关内容

影响：效果——答错 · 漏答

③ Attention Budget

注意力预算 · 成本问题：算得贵

n 个 token 两两计算

注意力 → $O(n^2)$

机制：两两注意力，计算量平方增长
预算被上下文长度吃掉

对策：控规模 · 分层缓存
KV 缓存复用

影响：成本——延迟 · 费用

LIM · Rot = 效果衰减 (漏答 / 答错) · Budget = 成本激增 (延迟 / 费用) ——上下文是稀缺资源，压缩是必需

注意力是稀缺资源——增加无关内容不是中性操作，会主动削弱有效信息的信号强度

上下文组成·分区·最左前缀——以 OpenCode 实测为例

一次请求上下文构成 (OpenCode 实测 · 无用户消息 · 合计 15,898 tok = 63,229 字符) · 按静态 / 动态分区：



Tools 13 个 = 上游 9 (bash · read · edit · search · write · webfetch · todowrite · question · skill) + Bocom 新增 4

Bocom 新增: memory_search (记忆检索) · project (跨项目) · task (子代理) · schedule (定时任务)

占比: System 30.2% vs Tools 68.9%——工具说明是每轮固定成本大头 (►Bocom Lite 第一刀) · token 估算: 中文×0.75 + 非中文×0.2416, 校准至 15,900

最左前缀原则 (Prompt Caching)

前缀字节级完全相同才命中 · 命中部分不计输入费 · 任何前缀改动 (含空格换行) 全失效 · cache_control 标记起点

实践: 可变内容置尾保前缀缓存 · CC fork 继承父工具数组同理 · 索引折叠同理

实测 68.9% 的 token 是工具说明——每轮重复计费的大头; 最左前缀稳定 = 缓存命中 = 省钱提速



上下文压缩：六项目怎么做

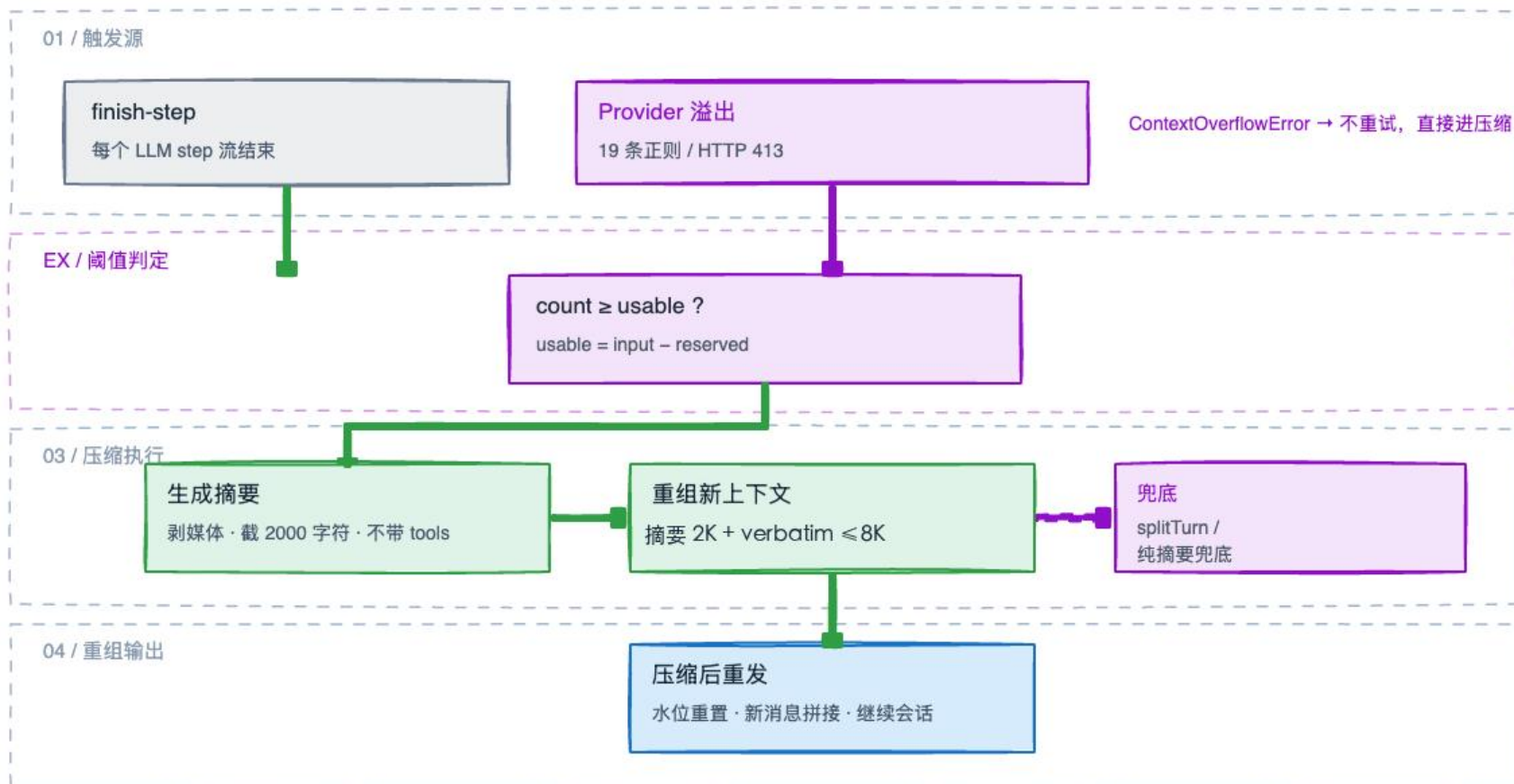
压缩的本质：对话快满时，把旧历史总结成一页笔记，只留最近原文——差异在何时动手、怎么删、留什么

项目	何时动手（触发）	怎么删（先免费的，再花钱的）	留什么 · 特色
Claude Code 五层递进瘦身	对话快满就动手 （窗口剩 13K）	先做不要钱的瘦身：限工具结果→剪旧历史→清工具输出→折叠细节 还不够，才叫子代理写 9 段总结	写总结复用主会话缓存——省钱 （不复用会浪费 98% 缓存）
DSH 两级 + 外置	双保险：每步检查一次 + 超窗报错再兜底	先把大工具输出剪成头尾片段 不够再 LLM 总结	超大输出（>50KB）挪外部文件 模型只看头尾预览 + 定位符
Codex LLM 总结	用到窗口九成就动手 （90% 阈值）	直接 LLM 总结压缩 两种统计口径可选：算全量 / 只算新增	长会话前缀固定时只盯新增 避免反复压缩
Pi 切点算法	窗口剩 16384 就动手 （保留 20000 原文）	只从完整消息处切 绝不断在工具调用中间	保证重放完整性 单文件 ≈850 行
Hermes 直接总结	超限就把历史压到一半 （×50%）	直接 LLM 总结（最简单） 无分级、无切点	压缩前先把记忆固化 （autoDream 配合）

共性三条：最近对话永远留原文 · 先免费瘦身再花钱总结 · 触发都是「事前检查 + 报错兜底」双保险
OpenCode 用 usable 预算线同时实现事前检查与溢出兜底——源码级流程见下一页

OpenCode 压缩机制：usable 预算 + 双路触发

OpenCode 上游实现：usable 预算线 · 八段摘要模板 · 近期原文保留窗口 · 溢出即压不重试



关键数据

$\text{usable} = \text{输入上限} - \text{预留}$ (例 $118\text{K} - 36\text{K} = 82\text{K}$) · $\text{verbatim} = \min(\text{usable} \times 25\%, 8\text{K})$ · 新上下文 = 摘要 2K + verbatim 起步

阈值对比：CC 窗口-13K · Codex 90% 双 scope · Pi -16384 · Hermes ×50% | OpenCode = usable 预算线 (上游原版)

上下文瘦身：通用三刀

为什么固定成本值得瘦身

系统提示 + 工具说明每轮原样重发 · 与当前任务无关的部分是纯开销
对应：token 成本 · 注意力预算（见第 8 页三约束）

原则：分级裁剪，不一刀切

按运行场景决定能力面：办公/编程/审查各有工具子集
判断标准：这个能力在本场景有没有用？没有就关，并给等价替代

第一刀 · 工具说明按场景裁剪

示例：bash 描述 7.3KB，办公场景剥离 git 提交 / PR 协议段落
示例裁到一半以内，示例数 6→2
保留：调用规则 · 截断行为 · 并行提示
失效引用改措辞 · 重复说明压一行

第二刀 · 系统提示渐进披露

同一份名单只发一遍（去 verbose 重复）
记忆索引折叠 → 一行提示，正文按需检索
示例 / 引号正反例压缩
文件本体无损 · 按需工具读取

两刀都砍在「每轮重复发送」的部分——只影响注入，文件与能力本体无损

工具说明占比常过半（见第 9 页实测 68.9%）——是瘦身收益最大、风险最小的一刀

收益

每轮固定 token 典型可降 30%~50%（视场景）
长会话按轮数线性放大节省
指令更聚焦 → 工具选择错误率下降
缓存友好的裁剪只影响尾部段

边界与代价（要讲清楚）

裁剪过度 → 模型失去关键上下文，宁少勿滥
被裁能力要给等价替代（如脚本/子代理）
双模式可切换：编程场景恢复全量工具面
灰度对比实测后再定默认值

瘦身≠删功能：按场景分级裁剪，等价替代兜底，收益与代价都讲清楚

第三章 · Memory System

生命周期

短期 → 中期 → 长期流转

保鲜与遗忘

冲突消除 · 评分门槛 · 主动淘汰

外置大脑

知识库与记忆的边界 · 两条实现路线

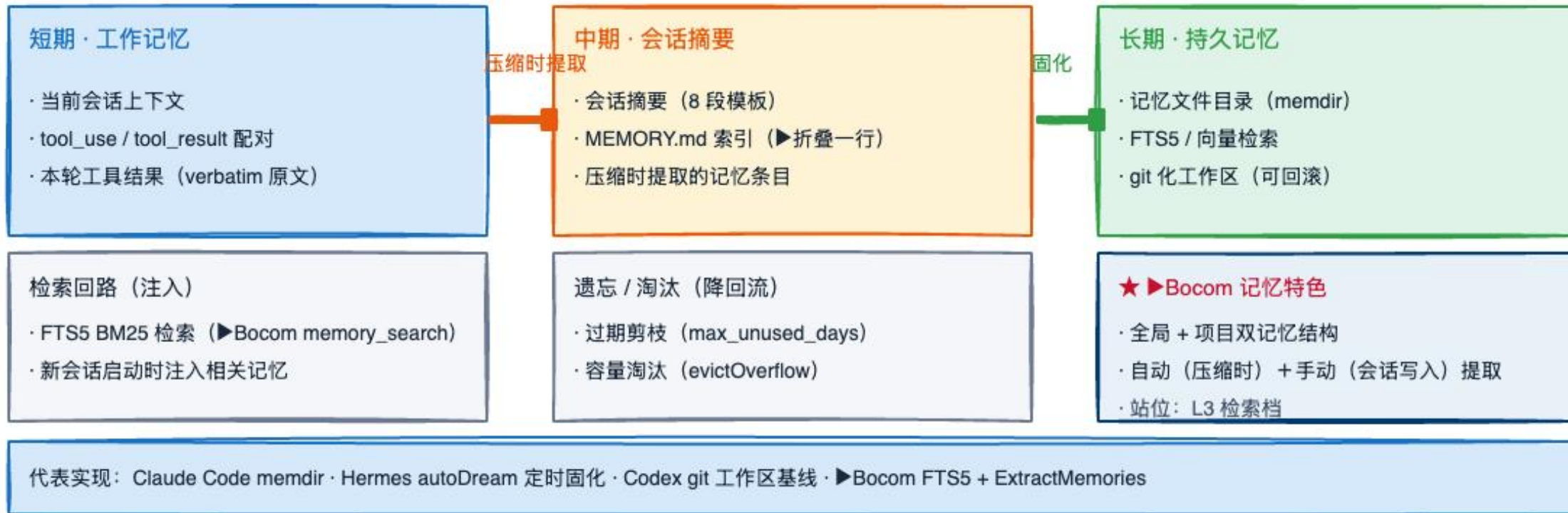
三种注入模式

全量注入 / 常驻索引 / 检索式 · 六项目对照

记忆生命周期：短期 → 中期 → 长期

工作记忆按需提炼为会话摘要，固化为持久记忆；检索回路按需注入；遗忘由剪枝与容量淘汰负责

Agent 记忆生命周期流转



▶ Bocom = L3 检索档: 索引折叠 (省 token) · FTS5 BM25 检索 (保能力) · 压缩时自动提取 (省调用)

对比

Claude Code memdir · Hermes autoDream 定时固化 · Codex git 工作区基线

记忆保鲜与遗忘：谁在管质量

留存记忆会过时、冲突、冗余——三家三种「质检」机制，全部来自源码级阅读

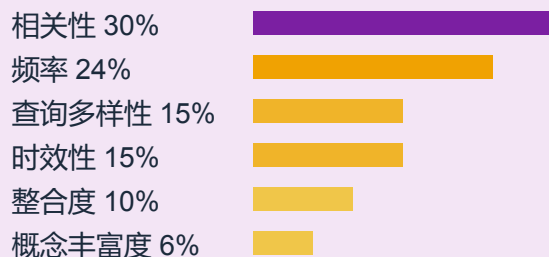
Claude Code · Auto Dream

- ① 触发：用户闲置 $\geq 24h$
(10 分钟扫描节流)
 - ② LLM 仲裁：消除矛盾 · 去除冗余
 - ③ 冲突以最新覆盖，旧内容自然淘汰
 - ④ 索引硬限： ≤ 200 行 · 每行 ≤ 150 字符
- 局限：关键词精确匹配，措辞不同即失效

保鲜方式：后台整合（闲置时运行）

保鲜流程：环境预检 → 时间门控 → 扫描

OpenClaw · 六维加权评分



硬门槛： < 0.8 分 · 召回 < 3 · 来源 < 3 → 丢弃

渐进晋升：Light Sleep 记短期信号
→ Deep Sleep 过滤后才写入 MEMORY.md
写前重读日志源文件，防误固化已删信息

通用设计准则（三家共性提炼）

- ① 保鲜靠 LLM 仲裁，不做无差别重写
消除矛盾 · 去冗余，冲突以最新覆盖
- ② 遗忘用「标记 → 计数 → 淘汰」，防误删
过期不立刻删，累计多次才落地
- ③ 写入前设门槛：评分/召回次数/来源数
低于门槛直接丢弃，宁缺毋滥
- ④ 写前重读源数据，防固化已删信息
以磁盘/日志为准，不以记忆为准

共同点：质量控制是独立环节

Codex · 两阶段管线的剪枝

Phase2: last_usage 超过 max_unused_days 窗口即剪枝
usage_count 高者优先保留 · git diff 基线可回溯

Hermes · 周期性保鲜

Periodic Nudge：每 10 轮自动发「该学了」信号
后台子代理异步整合 · MEMORY.md/USER.md 冻结快照

保鲜 = LLM 仲裁 + 评分门槛；遗忘 = 主动淘汰不靠自然漂移——没有质量控制，记忆系统会「越用越笨」

外置大脑与知识库：是什么 · 在哪 · 怎么实现

外置大脑 = 把大规模知识放模型外面：建索引、按需检索、注入上下文——解决「模型没见过 / 上下文装不下」的知识问题

在 Agent 中的位置与作用

知识库检索 Top-K

→

上下文组装 (System + 记忆 + 知识)

→

LLM 推理

作用：① 突破窗口限制——装不下的变成查得到的 ② 接入私有知识——企业文档 / 代码 ③ 降低幻觉——来源可溯源

记忆 (首选, 已建)

条目级 · 会话沉淀 · 几十条量级

关键词检索够用 (SQLite FTS5 + BM25)

知识库 (外置大脑)

文档 / 代码级 · 大体量 · 外部既有资产

需要语义检索 / 知识图谱 / 来源溯源

工作原理 (四步)

① 摄取 Ingestion
文档 / 代码 / PDF 切块

→

② 索引 Indexing
向量嵌入 / BM25 / 图谱

→

③ 检索 Retrieve
按相关性取 Top-K

→

④ 注入 Inject
拼进上下文 · 带溯源

路线 A · 云端 / RAG

代表：Mem0 · Letta (原 MemGPT) · 向量数据库

原理：文本 → 嵌入向量 → 相似度检索 Top-K

优势：语义理解强 · 大规模非结构化文档

代价：向量库运维 · 嵌入模型 · 网络可达

路线 B · 本地轻量

graphify (120k★)：代码/文档 → 本地知识图谱

tree-sitter AST 解析 · 无向量库 · 纯本地运行

claude-obsidian (15.1k★)：Obsidian 当知识库

BM25 检索 · 来源溯源 · plain Markdown 自持

选型提示：条目级记忆检索本地轻量即可 (走同类本地轻量技术路线) 大规模非结构化文档再上语义检索——定位是记忆检索，不是知识库 (详见下一页)

记忆注入的三种模式

记下来的内容怎么进上下文——常驻成本与召回率的权衡，六项目恰好落在三种模式上

① 全量注入

规则文件全文常驻上下文

CLAUDE.md / AGENTS.md 每轮随请求发送

优点：始终可见，不会漏

代价：成本随内容线性增长

内容多时挤占注意力预算

代表：多数项目的规则文件 · Pi 极简全文

② 常驻索引 + 按需读

只注入索引/摘要（几十 token/条）

正文留在磁盘，需要时整篇读取

优点：常驻成本小一个数量级

代价：多一次工具往返；索引质量决定

命中——description 要写「何时用」

代表：Claude Code memdir + MEMORY.md 索引

③ 检索式注入

FTS5/BM25（或向量）按查询取 Top-K

只把命中条目拼进上下文

优点：零常驻成本，规模可到万条

代价：措辞不匹配即漏召回

关键词检索 ≠ 语义检索

代表：SQLite FTS5 生态 · 知识库 RAG 同构

六项目站位（注入维度）

- Claude Code：memdir 目录 + 索引常驻 + Sonnet 选 5 条注入（②，半自动筛选）
- Hermes：预压缩刷写 + autoDream 定时固化（②+④ 自动化）；Codex：两阶段管线 + git 工作区基线
- OpenCode：规则文件全量注入（①），记忆靠插件生态扩展；Pi：极简全文（①）

选择顺序：先①小而精起步 → 内容变多切② → 规模化再③——每一步都保留上一步的召回兜底

第四章 · Agent Loop

循环思想

ReAct → TAOR · 三家工程风格

主循环

六步流水线：组装 → 推理 → 解析 → 守卫 → 执行 → 回注

优化手法

快照采样 · 无变化短路 · 写操作 opt-in · 慢环境兜底

原则

只有写工具才需要 baseline hash

Agent Loop: 从 ReAct 到 TAOR, 三种工程风格

两种表述一个思想; 主循环流水线见下一页——本页看思想映射与三家工程风格

TAOR 四阶段 → 通用实现映射

Think → LLM 流式调用 (组装后推理)

Act → 工具并行执行 (组内重叠 ≤ 10)

Observe → 结果回注 (tool_result)

Reflect → 摘要 / 压缩 (下一轮组装)

OpenCode 系: 主循环串行 · 工具层并行

串行 = 工程取舍: 可重放、易调试、状态确定

一个 step = 一次 LLM 调用 + 其工具执行

三家工程风格 (源码)

Claude Code: 六步 pipeline

autocompact fork 子代理生成 9 段摘要

DSH: ReactLoopAgent 本身是插件

循环可热替换 · 事件溯源 fork / 重放

Pi: 扁平双循环 · 25 个 hook 点

≈850 行单文件 · 极简但可挂载

OpenCode: 事件驱动 · 单线程串行

一轮对话的完整循环

用户发消息 → 组装上下文 (System + 记忆 + 知识) → LLM 思考并决定调工具 → 并行执行 → 结果回注 → 再思考 → ... → 最终回答 → 会话持久化

每轮都计数: token · 耗时 · 工具调用数——这就是可观测性章的数据来源 (先量化再动手)

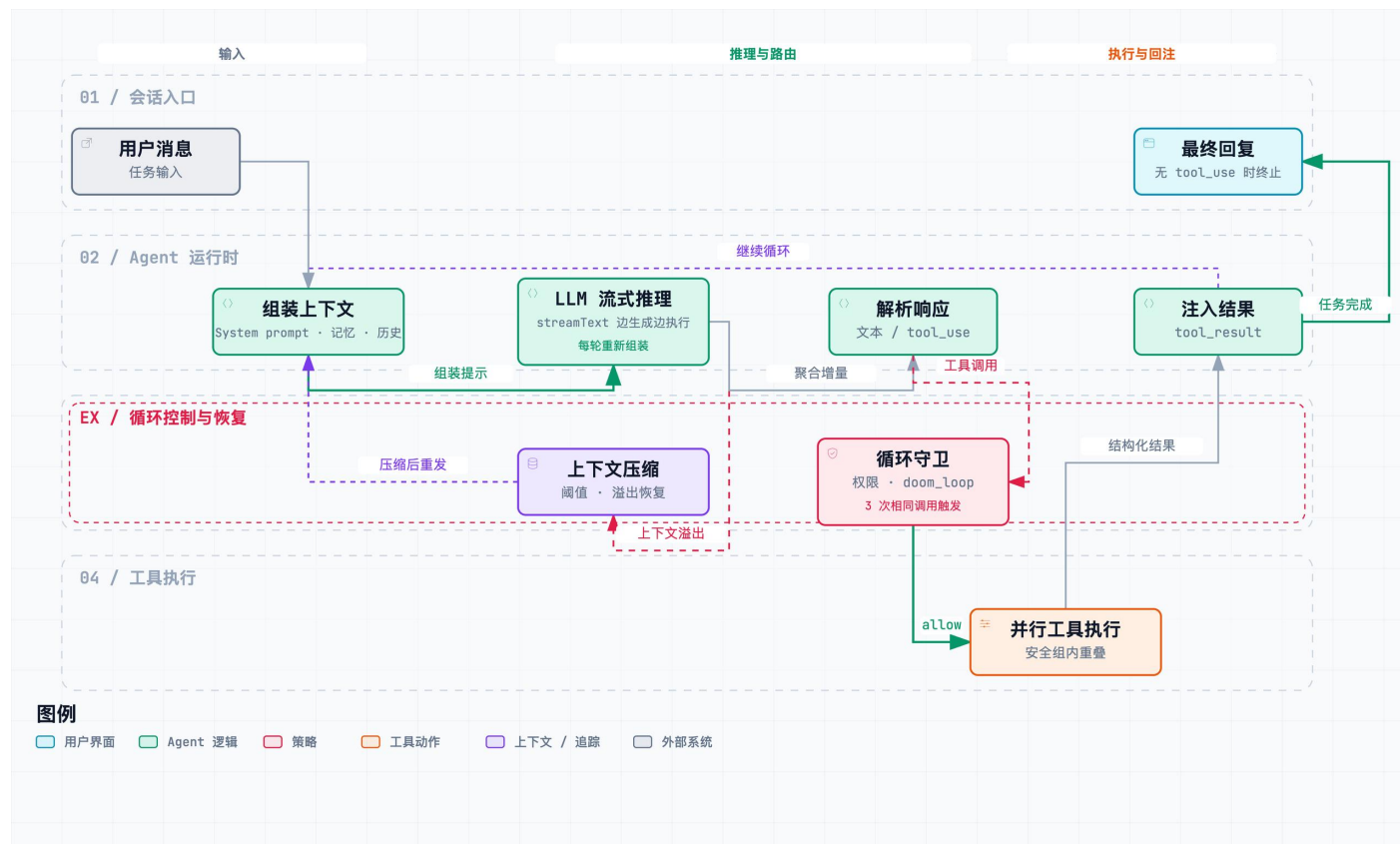
溢出旁路: 任一步 token 超限 → 直接进压缩 (流程见压缩章)

Prompt 只是建议, Harness code is law——

循环每一步都是确定性代码: 权限、执行、组装、压缩, LLM 只负责 Think

Agent Loop: 主循环通用形态

组装上下文 → 流式推理 → 解析 → 循环守卫 → 并行执行 → 结果回注；溢出走压缩后重发



对比

Claude Code 六步 pipeline · DSH 六态 TurnEndReason (max-tokens 粘性) · Pi 扁平双循环 25 hooks

Agent Loop：快照与观测的优化手法

快照 = git track/patch 支持会话回退 · 事件流 = 阶段化活动事件 · 慢环境 = 把无限等待变成超时降级

① 快照优化 · 三项

问题

快照点跟着每个工具走 → 调用量随 step 线性增长（数百次/轮）
读-only 工具也触发完整快照

三项优化

① per-message 边界

只在 message 边界调一次

② lastHash 缓存

diff-files 检查 → 无变化跳过

③ Writable Opt-In

只写文件才快照

git 调用 ↓1~2 个数量级

只读会话收尾开销归零
核心：只有写工具需要 baseline

fail-open 取舍：bash 默认不算写工具
少数 bash 写文件会话回退时不删新增
可接受

三项独立有效，叠加最优

② 阶段化事件流 · 8 阶段

8 阶段

thinking

generating

waiting

tool_input

tool_executing

compacting

summarizing

idle

29 类活动事件

异步发布

Bus + Effect.forkIn(scope) 独立 fiber

主循环额外延迟可忽略

TUI 表现

• 呼吸灯：当前阶段视觉化

• 瀑布流：每步耗时记录

• SubAgent 面板：最多 3 个 [BG]

关键

• forkIn + Bus 让观测零成本

• 异步发布不拖累主循环

③ 慢环境 · 超时兜底

三类症状

NFS/慢盘上 git 挂起 · 目录扫描卡顿 · 快照写盘慢
底层命令阻塞，应用层 catch 无效

三大手段

① 配置化超时

每个子进程套 AbortController

② 并行化

diff/ls-files 并行 · Grep fiber 化

③ 诊断埋点

13 处理点 · 50ms 阈值

配置超时

git 30s · snapshot 60s
grep 180s · hook 30s

核心

把「无限等待」变成
「超时后优雅降级」

关键

• 让“环境慢”变成“可观测、可配置”

• 不让慢操作拖垮整个会话

三项一体：快照（按写操作触发）+ 事件流（异步零阻塞）+ 超时降级（慢环境可控）

优化的前提是度量：先有结构化事件流，才有上述每一项改进的前后对比数据

21

可观测性与事件溯源

所有优化的前提是度量——把执行过程变成结构化事件流，问题才能定位、改动才有前后对比

事件流 (Event Stream)

把一次会话拆成阶段化事件：

thinking / generating / waiting /

tool_input / tool_executing /

compacting / summarizing / idle

每类活动带时长与 token 计数

异步发布：观测不阻塞主循环

TUI/IDE 据此渲染进度与耗时

用途：定位瓶颈 · 成本归因

事件溯源 (Event Sourcing)

更彻底：模型看到的每一条输入

都有日志对应 (visible $\Leftrightarrow$ logged)

会话 = 一串可重放的事件序列

由此获得三种能力：

- fork：从任意事件分叉新会话

- replay：确定性重放调试

- time-travel：回看任意一步

代表：DeepSeek Harness

遥测导出 (OpenTelemetry)

把内部事件流对接统一遥测标准：

- traces：请求级全链路追踪

- metrics：token / 延迟 / 成功率

Claude Code 官方支持 OTel 导出；

先有结构化事件，才谈得上遥测

企业侧收益：集中看板 · 告警 ·

跨团队成本归因与审计

关键：事件 schema 要稳定

可观测性三层次：事件流（看得见）→ 事件溯源（可重放）→ 遥测标准（管起来）

第五章 · Multi-Agent

为什么多 Agent

三个瓶颈 · 隔离才是本质

四种编排模式

Workflow / Supervisor / Hierarchical / Swarm

通信方式

请求响应 → 点对点 → 信箱 → A2A 协议

长寿命子代理 + 持久信箱

长寿命子代理 · 异步解耦收发

为什么要多 Agent：三个瓶颈

一个 Agent 干所有事，就像让一个人同时写代码、查资料、做质检——什么都干，什么都干不好

瓶颈① 上下文膨胀

笔记本写满了，重要的反而找不着

所有工具结果挤同一个窗口：

搜了 10 轮代码，每轮结果都留着
窗口被旧数据填满 → 新指令被稀释
触发压缩 → 早期细节丢失

解法：子代理独立窗口干脏活

大范围检索派出去，主循环只收结论
证据：CC microcompact 专为此设计

类比：你不会把所有草稿都留在桌上

瓶颈② 工具选择退化

钥匙太多，反而开错门

工具超 20 个，选择错误率上升：

每多一个工具 = 给选择题多一个选项
互相干扰 → 模型选错工具 → 浪费 token

解法：子代理只带相关小工具集

裁剪到场景最小集（Pi 只留 4 个）
Pi：只留 4 个（极简对照样本）

类比：不会把 20 把钥匙全挂一个环上

瓶颈③ 矛盾角色

又当运动员又当裁判

发散探索与严谨质检

塞同一个 system prompt 互相打架
写代码时想天马行空，验代码时步步紧逼

解法：质检子代理 maxTurns=1

最短寿命 · 只要独立性不要持续性
拒绝工具调用 = 任务失败（Sonnet 4.6）

类比：写手和审稿人不能是同一个人

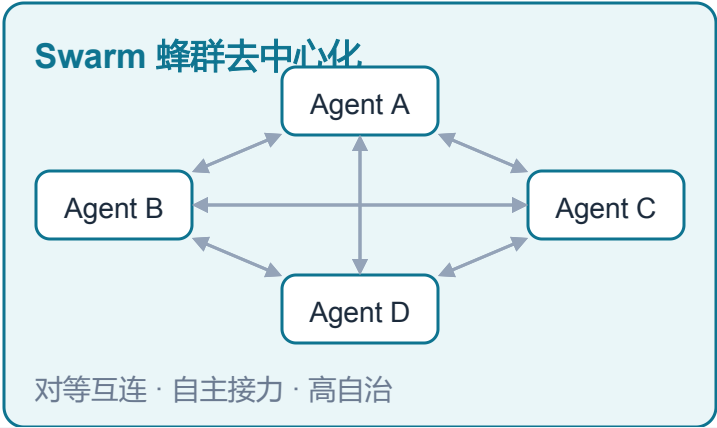
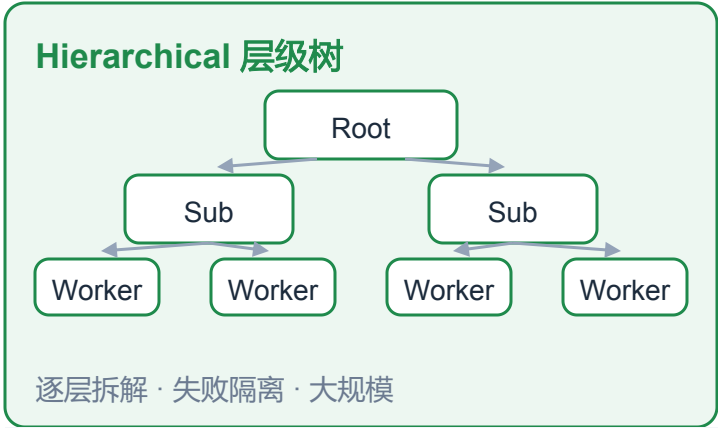
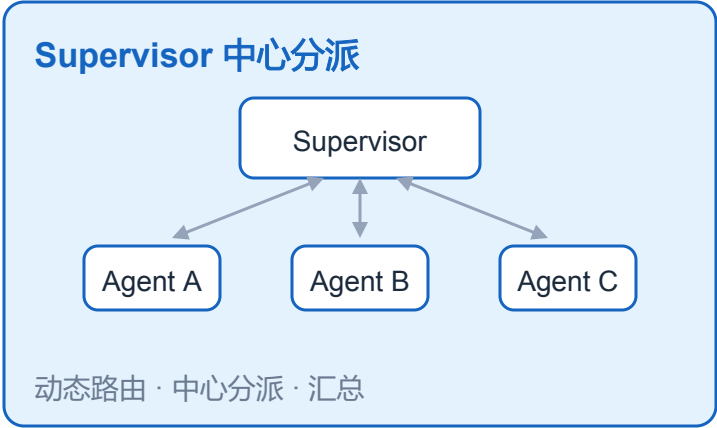
隔离才是多 Agent 的本质

每个子代理都有独立上下文窗口 + 独立工具集 + 独立权限——不是多脑袋，是多张干净的桌子

多 Agent 收益 = 上下文隔离 + 失败隔离 + 并行——落地模式见两页后的 Actor + Inbox

Multi-Agent：四种编排模式

四种编排范式的结构区别——连线方向与中心化程度即差异所在；右侧为编排核心要素



编排核心要素

① 任务拆解

Workflow 代码预写 · 其余动态拆解

② 子代理三种寿命

独立 —— 独立窗口，从零开始

fork —— 复制父上下文，延续思路

可续 —— 持久会话，随时追加消息

③ 结果回传两种范式

结构化报告 —— 父级只看结论不看过程

Inbox 异步 —— 发完即走，收发解耦

成本与容错

成本：子代理 = 独立上下文，token 倍增

容错：可重派 · 逐层隔离 · 冗余兜底

长寿命子代理 + 持久信箱 = Swarm 可控变体

选型关键

任务可预测 → Workflow | 需动态路由 → Supervisor | 大规模协作 → Hierarchical | 高自治需求 → Swarm

生产落点

CC Task+SendMessage / DSH 6 种 provider / Codex Thread Manager fork / Pi 无内置 | 长寿命子代理 + 持久信箱 = Swarm 可控变体

说明

OpenAI 官方 Swarm 无主 Agent——交给谁由**当前 Agent 自行 handoff**决；CC Agent Teams 由**Team Lead 统一分派**（Supervisor 变体）——分界 = 有无中心委派者

Agent 间通信：从请求响应到 A2A

四种模式逐层递进：通信能力上限决定编排自由度——从同步阻塞到标准化网络协议

① 父子请求-响应

父发起 → 阻塞等待 → 结果通过 tool_result 带回

同步：父循环被卡住，但语义最简单

无需独立通信设施；结果即 LLM 输出

适用：单层分解、结果必须立即使用

代表实现：CC Task / DSH fork（一次性）

局限：嵌套深时 token 线性倍增；长任务阻塞父循环，须等全部子完成才返回

子崩溃 → 父报错，无容错重试机制

风险：父等太久 → 超时 → 整轮失败

② 点对点消息

Agent 间直接互发消息；父不阻塞

框架只给传输通道，编排责任在模型

代表：CC SendMessage；Swarm 接力

无持久化：消息在内存，进程重启即丢失

代表实现：CC SendMessage / Swarm

局限：无确认机制；消息顺序无保证

无重试；模型需自行处理超时和重发

Swarm：接力式自主协作，全自治无中转

风险：消息丢失无感知；调试困难

③ 信箱 / 中转

消息入持久化信箱；发完即走，收发解耦

父子不互相阻塞；结果异步到达 Inbox

实现：内置 DB 专表（Inbox）持久化中转

架构：Actor registry 管等待 → 唤醒

长寿命 + 持久化中转 → Swarm 可控变体

优势：进程重启不丢消息；可跨轮次追踪

并行探索：派出去 → 主循环继续 → 按需读

适用：任务可拆 + 结果不需立即使用时

④ A2A 协议（业界标准）

Google 2025 推出 Agent2Agent

三核心：Agent 卡片（能力自描述）

+ 任务生命周期管理

+ 消息 / 工件交换

传输：JSON-RPC over HTTP / SSE

关键价值：跨厂商互操作（甲方不受绑定）

已入 Linux 基金会（2025.06 Google 捐赠）· 与 MCP 互补

趋势：Harness 内的 IPC → 标准化网络 API

四模式横向对比

通信能力逐层递进，越右越灵活，越左越简单

维度	① 同步	② P2P	③ Inbox	④ A2A
延迟模型	同步阻塞	异步	异步持久	网络异步
消息持久化	✗	✗	✓ DB	✓ 可选
结果回传	tool	流式	轮询	状态机
容错重试	✗	✗	✓ 框架	✓ 协议
互操作	同进程	同框架	同框架	✓ 跨厂商
实现复杂度	最低	低	中	高
代表项目	CC Task	SendMsg	Inbox 类	A2A
演进选型	Actor	—	Inbox	观望

关键洞察

①+② 同进程协作；跨机器须走 ③ 或 ④

③ 是 ④ 的内部基础；Inbox 可平滑升级 A2A

选型顺序：① 验证 → 瓶颈 → ③ → A2A 标准化

落地模式：长寿命 Actor + 持久 Inbox

基于 Actor 模型（异步消息 + 状态封装）：Actor 注册表 + Inbox 持久化数据表 · 对标 CC Task、DSH startContinuable

Actor 注册表

子代理登记为长寿命 Actor，父级拿到句柄即可继续自己的循环
registry 管理等待与唤醒：需要结果时再阻塞等待
对比：CC Task 一次性阻塞等报告

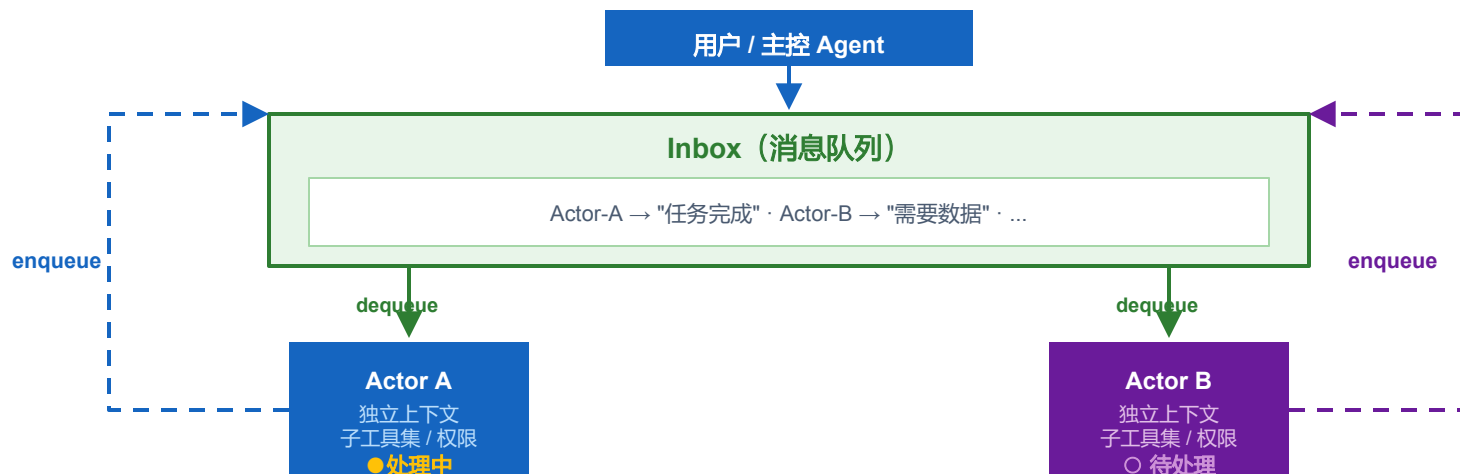
Inbox 收件箱

独立数据表（fork 专有 DB 表），消息持久化中转而非直连
父级发完消息即可继续工作，结果异步到达 Inbox
Swarm 可控变体：P2P 消息存在，但经中转

与两家对照

Claude Code：Task 阻塞等报告，SendMessage 提供 P2P
DSH：fork（一次性）/ startContinuable（持久可续）
可续+中转：Actor 长寿命 + Inbox 解耦收发

Actor + Inbox workflow（闭环）



并行不失控：消息经持久化中转，等待与唤醒由 registry 统一管理

典型用法：大范围代码检索 / 多方案调研派给子代理，主循环继续处理；子代理结果写入 Inbox，父级按需读取
适用判断：任务可拆且子任务结果不需立即使用时才值得异步化

第六章 · Tool Integration System

注册 · 并行 · 权限

fail-closed: 无匹配默认 ask · 循环守卫兜底

Skill

三层渐进式披露, 按需加载

MCP

标准化外挂协议 · 与 Skill 的选型判断

工具设计实践

description 写法 · 参数 schema · 错误面向模型

工具系统：注册 · 并行执行 · 权限

OpenCode 内置 16 工具 + 插件扩展 · Tool.define() 统一注册 · AI SDK streamText 并发执行 · 三层 Ruleset 权限

① 三层注册 + 插件接管

内置 16 + 文件 {tool,tools}/* + plugin.list()

插件同 ID 接管内置 (plugin-shadows-builtin)

writable 延续接管者 → 保留 revert 快照基线

② 按模型/Agent 动态裁剪

GPT → apply_patch · 主模型支持图片 → 隐藏 multimodal

Lite 模式: glob+grep → 合并为 search

activeTools 传给 streamText 决定模型可见集

③ 流式并发: streamText 单次输出多 tool-call → AI SDK 自动并行 invoke

- processor.runningTools Map 跟踪当前并发工具数 · 判定权在模型: 独立 call 自动并发, 依赖 call 自然串行
- activeTools 从 ToolRegistry.tools(model) 取出, 只暴露当前模型/Agent 应看到的工具

④ 循环守卫 (通用实践): 相同调用连续 N 次即拦

连续 3 次「同名工具 + 相同参数」调用 → 触发权限询问, 用户拒绝即中断——防止模型在死循环里烧 token

⑤ 权限: deny → ask → allow (permission/next.ts 518 行)

- 静态规则 → acceptEdits → 只读白名单 → ML 分类器, 逐层 evaluate (evaluate 15 行)



注册按场景裁剪 · 并发由模型判定 · 循环守卫 + permission fail-closed —— 三道保险

工具分类: 文件 read/write/edit · Shell bash · 检索 glob/grep · 网络 fetch · 任务 task/todo/question · 流程 skill/plan · 插件可扩展自定义工具

工具设计最佳实践：description 与 schema

工具即 API——模型靠 description 和参数 schema 决定「用不用、怎么用」，写法直接决定调用质量

description 写法四要素

- ① 做什么：一句话说清功能边界
- ② 何时用：触发场景 + 典型例子
- ③ 何时别用：与替代工具的分工（避免误选）
- ④ 行为约定：输出形态 · 截断规则 · 副作用

反例：只写「执行命令」——模型不知道何时该选它

正例：写明「读文件用 read；本工具只做检索定位」

参数 schema 设计原则

- ① 参数越少越好：每个参数都是一道选择题
- ② 命名与语义跨工具一致（path / pattern / mode）
- ③ 枚举代替自由字符串（mode: files|content）
- ④ 必填最少化：可选参数给默认值并写进描述

参数设计差 → 模型瞎填 → 调用失败 → 重试烧 token

合并同类工具（glob+grep→search）即减少选择错位

错误信息面向模型

工具报错是模型的「反馈信号」——要可行动：

「file not found」→ 附最接近的候选路径

「param invalid」→ 指出合法取值范围

错误进上下文，模型下一轮自纠错

只给人看的报错（堆栈）对模型是噪声

成本与确定性

- ① 输出截断写进契约：超限落盘 + 返回定位符
- ② 幂等声明：重复调用是否安全（写类工具标注）
- ③ 超时/取消语义：模型能预期中断行为

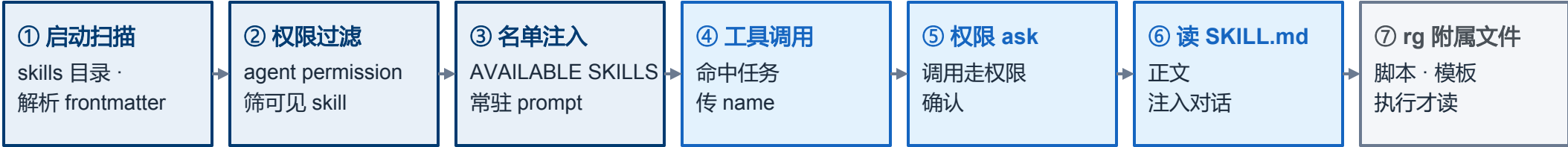
工具说明是「每轮重发的固定成本」——

写一半废话等于给每个工具加永久税

好工具 = 清晰分工的 description + 最小参数面 + 可行动的错误——模型的选择题越简单，调用越准

Skill 渐进式加载：三层披露 · OpenCode 加载流程

Skill = 教模型「怎么干」的知识包 · 只有被用到的 Skill 才完整加载（Anthropic 规范，四项目通用） · 下图 = OpenCode 实现流程



颜色 = 披露层级：■ 第①层 元数据常驻 ■ 第②层 正文按需 ■ 第③层 附属执行才读

第①层 · 元数据常驻

name + description 进上下文（几十 token）
模型靠 description 自主判断命中
自动发现：扫 ~/.claude/skills/ 等路径
常驻成本 = Skill 数量 × 几十 token
编写：description = 「做什么 + 何时用」+ 触发词

第②层 · 正文按需

命中才读 SKILL.md：frontmatter + Markdown
必填 name · description；正文 ≤500 行
结构：What I do / When to use / Steps
DSH 变体：三段渐进披露，段间按需
编写：细节外置附属 · 固定操作固化脚本 · 一职责

第③层 · 附属资源

模板 / 脚本 / 参考文件
相对路径引用，执行时才读取
一个 Skill 一个职责，避免大而全
Hermes 扩展：满足条件自动生成新 Skill
配套权限：external_directory 纳入可访问范围

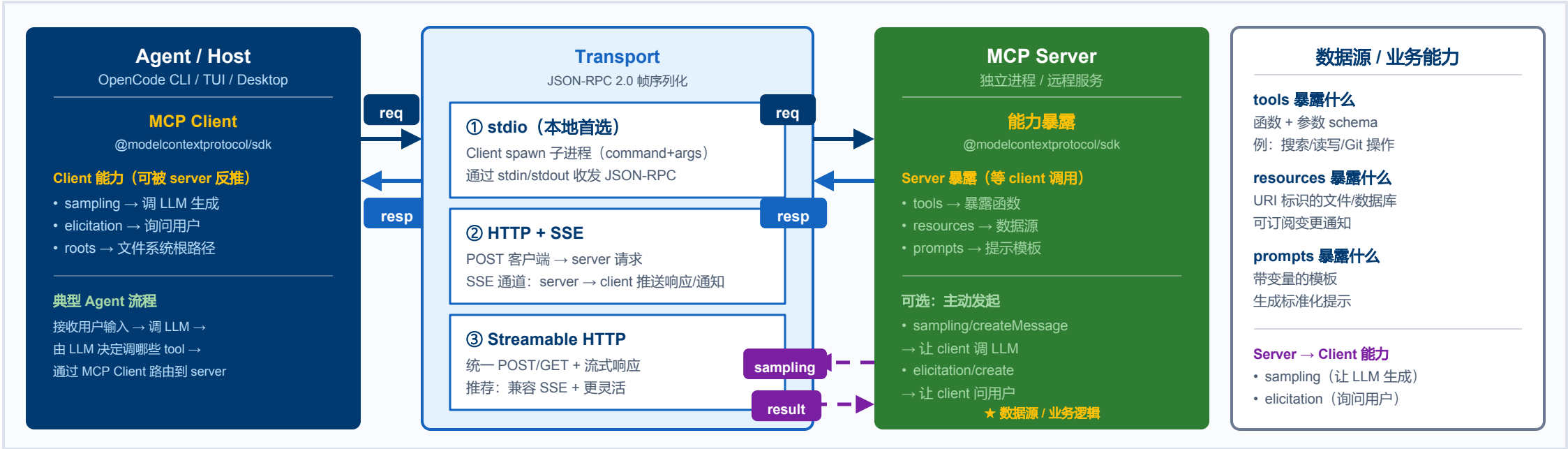
同族手法：同一份名单只发一遍（去 verbose 重复） · 记忆索引折叠同理——常驻只留索引，正文按需展开

上图加载流程即 OpenCode 原生实现：启动扫描 → 权限过滤 → fmt 注入 → skill 工具 → 权限 ask → 读正文 → rg 附属

信息完整性 × token 经济性：常驻只有索引（几十 token），正文与附属全部按需——用不到的 Skill 零成本

MCP: 双向 JSON-RPC 协议 · stdio / HTTP+SSE / Streamable HTTP

Anthropic 2024.11 开源 · mcp/index.ts 999 行 · Server 暴露 tools/resources/prompts · Client 提供 sampling/elicitation/roots



MCP Server 暴露什么能力?

tools · 让 Agent 能调用的函数

类比: API 端点。每个 tool 有名字、描述、参数 schema。

例: `github.create_issue(repo, title, body)`

resources · 只读的数据源

类比: 文件系统 / 数据库视图。用 URI 标识。

例: `file:///repo/README.md` · 可订阅变更

prompts · 参数化的提示模板

类比: 邮件模板。Server 提供模板 (含变量)。

例: `code_review.pr(diff, focus_area)`

握手生命周期

① Client → Server: initialize (声明协议版本 + client 能力) ② Server → Client: 返回 server 能力

③ Client → Server: notifications/initialized ④ 进入正常操作 (双方均可发 request/notification)

JSON-RPC 2.0 报文

请求: `{ "jsonrpc": "2.0", "id": 1, "method": "tools/call", "params": { "name": "...", "arguments": {...} } }`

通知: `{ "jsonrpc": "2.0", "method": "notifications/...", "params": {...} }` (无 id 不需响应)

第七章 · Agent 自进化

RSI 双路径

改模型权重 vs 改 Harness 代码

三层路径

Context → Workflow → Harness 本身

记忆阶梯

L1 手写 → L5 自我管理 · 六项目站位

另两条路线

DSH 重构式 · RL 参数化

业界视野：自进化 · RSI 与 Harness 双路径

RSI (Recursive Self-Improvement, 递归自我改进) + 翁荔 2026.07 《Harness Engineering for Self-Improvement》 · LangChain: 换 Harness, 52.8% → 66.5%

路径 1 · RSI (递归自我改进)	模型方向	路径 2 · Harness Engineering (驾驭代码自改)	代码方向
定义: AI 系统迭代修改自身 (含权重), 每一代比上一代更强 关键概念: • I.J. Good (1965) "Intelligence Explosion" — 智能超线性提升 • Seed AI — 起点, 能力可启动 RSI 循环 • Takeoff (起飞) — soft / medium / hard 三类速度假设 风险: • 对齐问题 (alignment) — 目标与人类意图偏离 • 失控问题 (control) — 安全不能放进化循环内		定义: 不改模型权重, 让 coding agent 优化 Harness (代码/MCP/Skill) 关键数据: • DGM 进化搜索: SWE-bench 20% → 50% (已超人工) • LangChain Terminal Bench: 52.8% → 66.5% • 模型权重一个字节都不改, 纯 Harness 优化 优势: • 可控: 每步 diff 可 review, 可回滚 • 安全: 权限/redline 不进入自进化循环	崔添翼: 两条路径 同样可能 出成果

翁荔 · Harness 自进化三层路径 (深度递增)



关键概念: 补偿面迁移 (翁荔) + RSI 安全红线

- Harness 组件的存在理由 = 模型当前做不到什么 · 模型越强 → 组件重心迁移
- RSI 红线: 权限/安全层必须留在自进化循环外 (fail-closed 纪律)

关键不在 Harness 多厚, 而在能否跟上模型能力的变化

记忆进化五档：从手写到自管理

六项目记忆自进化能力对照（细节见 Harness Dev Book §7.8）· 自上而下自动化程度递增：L1 纯人工 → L5 自管理（层次高低 ≠ 能力强弱，仅表示自动化程度）



关键洞察：自进化不止“长出来”，是 L1→L5 的能力梯度

- L1→L2: 人类维护 → 人工添加（量变） · L2→L3: AI 选 → AI 写（质变，事件触发）
- L3→L4: 同步提取 → 预压缩刷写（自动化） · L4→L5: 定时固化 → 自我管理技能（前沿研究）

L3 检索式与 Harness 自进化（路径 2）方向一致 —— 都是 L1→L5 渐进

六项目自进化能力对照

Harness 方向 vs 模型方向的渐进 · 不一定越高越好——DSH/Pi 刻意停在 L1（押注可组合性/极简）

Hermes	L4 自动刷写	Codex	L3 检索化	Claude Code	L4 自动刷写	OpenCode（无内置）	L3 检索化	DSH	L1（刻意不做）	Pi	L1 极简
--------	---------	-------	--------	-------------	---------	---------------	--------	-----	----------	----	-------

自进化的另外两条路：DSH 重构式 · RL 参数化

生长式之外的两条路线 · 三种取舍：生长式 / 重构式 / 参数化 —— 评估器成熟前前两种更稳（来源：Harness Dev Book §7）

DSH：重构式自进化	RL：参数化自进化
换零件不换长出来	写进模型权重
核心思路：押注结构可组合 + 日志可重构 三大机制 ① Agent Loop 本身是插件 · 可整体替换 不修改代码，只换循环模块 ② 事件溯源 · 任何会话可 fork / 重放 事件溯源 = 会话可分叉、可重做 ③ 改进不靠长出来 · 靠换零件 每个组件可单独升级 / 替换 刻意不做：记忆 / 技能自进化 押注可组合性优先	核心思路：用 RL 把验证过的行为写进模型权重 两种主流算法 GRPO · 8~16 采样组内相对比较 不需要奖励模型（reward model free） GiGPO · 锚定状态聚合 + 步进优势加权 解决 50+ 步长任务稀疏奖励问题 数据来源 · 不用用户对话 真实目的是知识蒸馏 局限：评估器成熟度决定成败 评估器不成熟前慎用

三种取舍：生长式 · 重构式 · 参数化		
生长式 长出记忆 / 技能 代表：Hermes · ACE/MCE 适合：探索期	重构式 换零件而非长出来 代表：DSH 适合：稳定期（押注可组合）	参数化 写进模型权重 代表：GRPO · GiGPO 适合：评估器成熟期

六项目自进化路线站位（现状）

Hermes	★ 生长 + RL	DSH	重构式	Codex	工程化管线	Claude Code	半自动	生长式框架 ★ ACE/MCE 路线	Pi	极简取向
--------	-----------	-----	-----	-------	-------	-------------	-----	--------------------	----	------

Outlook: Harness 的演进方向

相关前沿方向

Loop Engineering (Outer Loop 编排)

Graph Engineering: Loops → Graphs, 控制流为 DAG

节点干活 · 边做路由 · 控制流显式化

一个观察: 补偿面迁移

Harness 组件存在理由 = 模型当前做不到什么

参考: Hermes 自进化双路径 (生长 + RL)



模型越强, 补偿组件的重心随之迁移 (Anthropic 已在拆解部分早期组件)

方向: 让确定性约束更完善、更易维护 · 感谢聆听, 欢迎提问